NumPy

Revision Sheet — Quick Reference

What is NumPy?

NumPy (Numerical Python) is the core library for numerical computing in Python.

It provides fast, memory-efficient arrays and vectorized math operations — no loops needed.

Standard Python lists are slow for math. NumPy arrays are up to 50x faster.

It is the foundation of data science, machine learning, and analytics in Python.

Why it matters:

- Vectorized ops → no for-loops, clean readable code
- Built-in functions: sum, mean, std, min, max, sqrt, percentile...
- Boolean masking for instant data filtering
- Essential for pandas, scikit-learn, TensorFlow, and every ML stack

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
result = arr * 2   # [2 4 6 8 10] — no loop needed
```

01 Array Creation

Create NumPy arrays from scratch using np.array() or built-in helpers.
Use zeros/ones for initialised arrays, arange for ranges, linspace for even spacing.

```
arr   = np.array([1, 2, 3, 4, 5])     # 1D array
arr2d = np.array([[1,2,3],[4,5,6]])    # 2D array (matrix)

np.zeros(5)         # [0. 0. 0. 0. 0.]
np.ones(5)          # [1. 1. 1. 1. 1.]
np.arange(0, 10, 2) # [0 2 4 6 8]
np.linspace(0, 1, 5)# [0.   0.25 0.5  0.75 1.  ]
```

02 Array Properties

Check shape, size, dimensions, and dtype before working with any array.
.shape and .dtype are the first things to check when debugging data issues.

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

arr.shape   # (2, 3)  — 2 rows, 3 cols
arr.ndim    # 2       — number of dimensions
arr.size    # 6       — total elements
arr.dtype   # int64   — data type
```

03 Indexing & Slicing

Access specific elements or sub-arrays. Index starts at 0. Negative index counts from end.
2D arrays use [row, col] syntax. Slicing follows start:stop (stop not included).

```
arr = np.array([1, 2, 3, 4])
arr[0]    # 1   |  arr[-1]   # 4
arr[1:3]  # [2 3]  |  arr[::-1] # [4 3 2 1]

arr2d = np.array([[1,2,3],[4,5,6],[7,8,9]])
arr2d[0, 1]      # 2  — row 0, col 1
arr2d[0:2, 1:3]  # [[2,3],[5,6]]
```

04 Math Operations

NumPy does element-wise math on entire arrays — no loops. Operations broadcast across arrays.
Built-in aggregation: sum, mean, min, max, std. Use axis=0 (column) or axis=1 (row) on 2D.

```
arr = np.array([10, 20, 30, 40, 50])
arr + 5      # [15 25 35 45 55]
arr * 2      # [20 40 60 80 100]

np.sum(arr)  # 150  |  np.mean(arr) # 30.0
np.std(arr)  # 14.14  — spread from mean
np.sum(arr2d, axis=0)  # column-wise sum
```

05 Reshape & Flatten

Change array shape without changing data. Total elements must stay the same.
Flatten converts any nD array back to 1D — common before feeding data to ML models.

```
arr = np.arange(1, 7)   # [1 2 3 4 5 6]
arr.reshape(2, 3)       # [[1 2 3][4 5 6]]

a = np.array([[1,2,3],[4,5,6]])
a.flatten()             # [1 2 3 4 5 6]
```

06 Boolean Masking

Filter array elements by condition — no loops. Returns only elements where condition is True.
Combine conditions with & (AND) and | (OR). Most used feature in real data analysis.

```
arr = np.array([10, 15, 20, 25, 30])
arr[arr > 20]                  # [25 30]
arr[(arr > 15) & (arr < 30)]   # [20 25]

salaries = np.array([20000, 45000, 60000, 35000])
salaries[salaries > 40000]     # [45000 60000]
```

07 np.where

Vectorized if-else. Replaces values or assigns labels based on a condition.

```
arr = np.array([10, 45, 23, 67, 5, 89])
np.where(arr > 30, 'high', 'low')
# ['low' 'high' 'low' 'high' 'low' 'high']

np.where(arr > 30, arr, 0)
# [ 0  45   0  67   0  89]
```

08 Percentile

np.percentile(arr, p) — value below which p% of data falls.
Q1=25th, Median=50th, Q3=75th. Values outside 5th-95th range are often flagged as outliers.

```
arr = np.array([10,20,30,40,50,60,70,80,90,100])

np.percentile(arr, 25)  # 32.5  — Q1
np.percentile(arr, 50)  # 55.0  — Median
np.percentile(arr, 75)  # 77.5  — Q3

# Detect outliers
lo = np.percentile(arr, 5)
hi = np.percentile(arr, 95)
outliers = arr[(arr < lo) | (arr > hi)]
```

09 Correlation

np.corrcoef(a, b) — how strongly two variables move together.
Result: +1 = perfect positive, -1 = perfect negative, 0 = no relation. Read corr[0][1].

```
study_hours = np.array([1, 2, 3, 4, 5, 6, 7])
marks       = np.array([35,45,50,60,70,75,85])

corr = np.corrcoef(study_hours, marks)
print(corr[0][1])   # 0.99 — very strong positive

# Practice example
temp = np.array([30,32,35,38,40,42,45])
ice_cream = np.array([200,220,250,300,320,350,400])
np.corrcoef(temp, ice_cream)[0][1]  # ~0.99
```

Topics Covered

Array Creation • Properties • Indexing • Math • Reshape • Masking • where • Percentile • Correlation